

Question No:5

Question description

A beautiful *code* of a tree of n nodes is a sequence of $n-2$ integers that uniquely specifies the structure of the tree.

The code is constructed as follows: As long as there are at least three nodes left, find a leaf with the smallest label, add the label of its only neighbour to the code, and remove the leaf from the tree.

Given a beautiful code of a tree, your task is to construct the original tree.

Constraints

- $3 \leq n \leq 2 \cdot 10^5$
- $1 \leq a, b \leq n$

Input

The first input line contains an integer n : the number of nodes. The nodes are numbered $1, 2, \dots, n$.

The second line contains $n-2$ integers: the beautiful code.

Output

Print $n-1$ lines describing the edges of the tree. Each line has to contain two integers a and b : there is an edge between nodes a and b . You can print the edges in any order.

✓ Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5
2 2 4
```

EXPECTED OUTPUT

```
1 2
3 2
2 4
4 5
```

Test Case 2

INPUT (STDIN)

```
5
3 1 2
```

EXPECTED OUTPUT

```
4 3
3 1
1 2
2 5
```

Code:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  typedef struct
4  {
5      int *data;
6      int size;
7  } MinHeap;
8
9  void swap(int *a, int *b)
10 {
11     int temp = *a;
12     *a = *b;
13     *b = temp;
14 }
15
16 void push(MinHeap *heap, int value)
17 {
18     int i = heap->size++;
19     heap->data[i] = value;
20
21     while(i > 0)
22     {
23         int parent = (i - 1) / 2;
24
25         if(heap->data[parent] <= heap->data[i])
26             break;
27
28         swap(&heap->data[parent], &heap->data[i]);
```

```
29         i = parent;
30     }
31 }
32
33 int pop(MinHeap *heap)
34 {
35     int result = heap->data[0];
36
37     heap->data[0] = heap->data[--heap->size];
38
39     int i = 0;
40
41     while(1)
42     {
43         int left = 2 * i + 1;
44         int right = 2 * i + 2;
45         int smallest = i;
46
47         if(left < heap->size &&
48            heap->data[left] < heap->data[smallest])
49         {
50             smallest = left;
51         }
52
53         if(right < heap->size &&
```

```
54         heap->data[right] < heap->data[smallest])
55     {
56         smallest = right;
57     }
58
59     if(smallest == i)
60         break;
61
62     swap(&heap->data[i], &heap->data[smallest]);
63     i = smallest;
64 }
65
66 return result;
67 }
68
69 int main()
70 {
71     int n;
72
73     scanf("%d", &n);
74
75     int *code = malloc((n - 2) * sizeof(int));
76     int *degree = calloc(n + 1, sizeof(int));
77
```

```

78      /* Read Prüfer code */
79      for(int i = 0; i < n - 2; i++)
80      {
81          scanf("%d", &code[i]);
82          degree[code[i]]++;
83      }
84
85      /* Create min-heap */
86      MinHeap heap;
87      heap.data = malloc((n + 1) * sizeof(int));
88      heap.size = 0;
89
90      /* All nodes with degree 1 are leaves */
91      for(int i = 1; i <= n; i++)
92      {
93          if(degree[i] == 0)
94          {
95              /* Nodes not appearing in Prüfer code have degree 1 */
96              push(&heap, i);
97          }
98      }
99
100     /*
101     Process the Prüfer code
102     */
103     for(int i = 0; i < n - 2; i++)

```

```

104     {
105         int leaf = pop(&heap);
106         int neighbour = code[i];
107
108         printf("%d %d\n", leaf, neighbour);
109
110         degree[neighbour]--;
111
112         if(degree[neighbour] == 0)
113         {
114             push(&heap, neighbour);
115         }
116     }
117
118     /* Two nodes remain */
119     int u = pop(&heap);
120     int v = pop(&heap);
121
122     printf("%d %d\n", u, v);
123
124     free(code);
125     free(degree);
126     free(heap.data);
127
128     return 0;
129 }

```

Output:

Test Case-1:

```

5
2 2 4
1 2
3 2
2 4
4 5

```

Test Case-2:

```

5
3 1 2
4 3
3 1
1 2
2 5

```

Keshav Kumar Rauniyar

CH.SC.U4CSE25236